

Переменные, типы данных и операторы

Полное обучение на примере Python 3. Концепции почти одинаковы в других языках (JavaScript, C#, C++) — меняется только синтаксис.

1. Переменные

Что такое переменная

Переменная — это **имя, привязанное к значению в памяти**. Представь подписанную коробку: на ней написано `score`, внутри лежит `100`.

```
python

score = 100
player_name = "Arsi"
is_alive = True
```

Знак `=` здесь не означает «равно». Он означает **присвоить**: взять значение справа и привязать к нему имя слева.

Значение можно менять

```
python

hp = 100
hp = hp - 25  # сначала вычисляется правая часть (100 - 25), потом результат
               записывается в hp
print(hp)     # 75
```

Правила имён

Можно	Нельзя
<code>player_hp</code> , <code>score2</code> , <code>_temp</code>	<code>2score</code> (начинается с цифры)
латиница, цифры, <code>_</code>	<code>player-hp</code> (дефис), <code>my var</code> (пробел)
	<code>class</code> , <code>if</code> , <code>for</code> (зарезервированные слова)

Хорошие привычки:

- В Python имена пишут в стиле `snake_case`: `max_health`, `enemy_count`.
- Имя должно объяснять смысл: `spawn_delay` лучше, чем `x`.

- Регистр важен: `Score` и `score` — это разные переменные.
- Константы по договорённости пишут заглавными буквами: `MAX_LEVEL = 50`.

Несколько присваиваний сразу

```
python
x, y = 10, 20      # x = 10, y = 20
x, y = y, x         # обмен значений: теперь x = 20, y = 10
a = b = c = 0       # всем трём присваивается 0
```

Динамическая типизация

В Python тип хранится у **значения**, а не у переменной. Одной и той же переменной можно присвоить значение другого типа:

```
python
data = 5           # int
data = "пять"      # теперь str
```

Python это разрешает, но лучше так не делать: код становится запутанным. В C# или C++ так нельзя: там тип переменной задаётся один раз (`int data = 5;`).

2. Типы данных

2.1. Основные (простые) типы

Тип	Что хранит	Пример
<code>int</code>	целые числа	<code>42</code> , <code>-7</code> , <code>0</code>
<code>float</code>	дробные числа	<code>3.14</code> , <code>-0.5</code> , <code>1e3</code> (= 1000.0)
<code>str</code>	строки (текст)	<code>"привет"</code> , <code>'a'</code>
<code>bool</code>	логическое значение	<code>True</code> , <code>False</code>
<code>NoneType</code>	«ничего», отсутствие значения	<code>None</code>

Тип значения можно узнать функцией `type()` :

```
python

print(type(42))      # <class 'int'>
print(type(3.0))     # <class 'float'>
print(type("hi"))    # <class 'str'>
print(type(True))    # <class 'bool'>
print(type(None))    # <class 'NoneType'>
```

int — целые числа

В Python целые числа могут быть сколь угодно большими: `2 ** 1000` посчитается без ошибки.

float — дробные числа

Есть важная ловушка — **неточность дробных чисел**:

```
python

print(0.1 + 0.2)      # 0.30000000000000004
print(0.1 + 0.2 == 0.3) # False!
```

Причина в том, что компьютер хранит дроби в двоичной системе, и `0.1` в ней точно не записывается. Поэтому дробные числа сравнивают с допуском:

```
python

import math
print(math.isclose(0.1 + 0.2, 0.3)) # True
```

str — строки

```
python

name = "Arsi"
greeting = 'Привет'
multi = """Строка
в несколько
строк"""
```

Основные операции со строками:

```
python

s = "Python"
print(len(s))      # 6 — длина строки
print(s[0])        # 'P' — индексы начинаются с 0
print(s[-1])       # 'n' — отрицательный индекс считает с конца
print(s[0:3])      # 'Pyt' — срез: от 0 до 3, не включая 3
print(s.upper())   # 'PYTHON'
print(s.lower())   # 'python'
print("Py" in s)   # True — есть ли подстрока

# f-строки — лучший способ вставить значения в текст
hp = 75
print(f"У игрока {name} осталось {hp} HP")
```

 Строки **неизменяемы**: `s[0] = "J"` вызовет ошибку. Нужно создать новую строку: `s = "J" + s[1:]`.

bool — логический тип

Всего два значения: `True` и `False`. Чаще всего это результат сравнения:

```
python

print(5 > 3)      # True
```

Truthy / Falsy. В условиях любое значение воспринимается как истинное или ложное. Ложными (falsy) считаются: `False`, `0`, `0.0`, `""` (пустая строка), `None`, `[]`, `{}`, `()`, `set()`. Всё остальное считается истинным.

```
python

inventory = []
if not inventory:
    print("Инвентарь пуст")
```

None — «ничего»

Означает, что значения нет (ещё не задано, не найдено).

```
python

target = None
if target is None:
    print("Цели нет")
```

2.2. Коллекции (составные типы)

Тип	Описание	Изменяемый?	Пример
<code>list</code>	упорядоченный список	✓ да	<code>[1, 2, 3]</code>
<code>tuple</code>	упорядоченный кортеж	✗ нет	<code>(10, 20)</code>
<code>dict</code>	словарь «ключ → значение»	✓ да	<code>{"hp": 100}</code>
<code>set</code>	множество уникальных элементов	✓ да	<code>{1, 2, 3}</code>

list — список

```
python

items = ["меч", "щит", "зелье"]
items.append("лук")      # добавить в конец
items[0] = "топор"      # заменить элемент
items.remove("щит")     # удалить по значению
print(items)            # ['топор', 'зелье', 'лук']
print(len(items))       # 3
```

tuple — кортеж

Похож на список, но изменить его нельзя. Подходит для фиксированных наборов значений, например координат.

```
python

pos = (10, 20)
x, y = pos              # распаковка
# pos[0] = 5            # ошибка: кортеж неизменяем
```

dict — словарь

```
python

player = {"name": "Arsi", "hp": 100, "level": 3}
print(player["hp"])     # 100
player["hp"] -= 10      # изменить значение
player["gold"] = 50     # добавить новый ключ
print(player.get("mana", 0)) # 0 — безопасное чтение со значением по умолчанию
```

set — множество

Хранит только уникальные значения, порядок не гарантирован.

```
python

tags = {"rpg", "roguelike", "rpg"}
print(tags)    # {'rpg', 'roguelike'} – дубликат исчез
a = {1, 2, 3}
b = {2, 3, 4}
print(a & b)    # {2, 3} – пересечение
print(a | b)    # {1, 2, 3, 4} – объединение
print(a - b)    # {1} – разность
```

2.3. Изменяемые и неизменяемые типы

Это одна из самых важных тем.

- **Неизменяемые:** `int`, `float`, `str`, `bool`, `None`, `tuple`.
- **Изменяемые:** `list`, `dict`, `set`.

Ловушка — две переменные, которые указывают на один и тот же объект:

```
python

a = [1, 2, 3]
b = a          # b – не копия, а второе имя для ТОГО ЖЕ списка
b.append(4)
print(a)       # [1, 2, 3, 4] – a тоже изменился!

c = a.copy()   # настоящая копия
c.append(5)
print(a)       # [1, 2, 3, 4] – a не тронут
```

С неизменяемыми типами такой проблемы нет:

```
python

x = 5
y = x
y += 1
print(x)    # 5
```

2.4. Преобразование типов

```
python

int("42")      # 42
int(3.99)      # 3 – дробная часть отбрасывается, а не округляется
float("3.5")    # 3.5
str(100)       # '100'
bool(0)        # False
bool("abc")    # True
list("abc")    # ['a', 'b', 'c']
round(3.6)     # 4
```

Типичная ситуация: `input()` всегда возвращает строку.

```
python
age = input("Возраст: ")    # '25' — это строка!
age = int(age)              # теперь число
print(age + 1)              # 26
```

Ошибки преобразования:

```
python
int("abc")                 # ValueError
"5" + 5                    # TypeError: строку и число нельзя сложить
"5" + str(5)               # '55'
int("5") + 5               # 10
```

3. Операторы

3.1. Арифметические

Оператор	Действие	Пример	Результат
<code>+</code>	сложение	<code>7 + 2</code>	<code>9</code>
<code>-</code>	вычитание	<code>7 - 2</code>	<code>5</code>
<code>*</code>	умножение	<code>7 * 2</code>	<code>14</code>
<code>/</code>	деление (всегда даёт float)	<code>7 / 2</code>	<code>3.5</code>
<code>//</code>	целочисленное деление	<code>7 // 2</code>	<code>3</code>
<code>%</code>	остаток от деления	<code>7 % 2</code>	<code>1</code>
<code>**</code>	степень	<code>2 ** 3</code>	<code>8</code>

Полезные приёмы:

```
python
n % 2 == 0                # проверка на чётность
total_sec = 125
minutes = total_sec // 60  # 2
seconds = total_sec % 60   # 5
index = (index + 1) % 4    # циклический счётчик 0,1,2,3,0,1...
```

Операторы работают и со строками и списками:

python

```
"ab" + "cd"      # 'abcd'
"ha" * 3         # 'hahaha'
[1, 2] + [3]     # [1, 2, 3]
[0] * 4          # [0, 0, 0, 0]
```



Особенность: `-7 // 2` даёт `-4`, а не `-3`, потому что округление идёт вниз, в сторону минус бесконечности.

3.2. Операторы присваивания

Оператор	Равносильно
<code>x = 5</code>	присвоить
<code>x += 3</code>	<code>x = x + 3</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 3</code>	<code>x = x / 3</code>
<code>x //= 3</code>	<code>x = x // 3</code>
<code>x %= 3</code>	<code>x = x % 3</code>
<code>x **= 3</code>	<code>x = x ** 3</code>



В Python нет операторов `x++` и `x--` (они есть в C/JS). Пиши `x += 1`.

3.3. Операторы сравнения

Результат сравнения всегда `True` или `False`.

Оператор	Значение
<code>==</code>	равно
<code>!=</code>	не равно
<code>></code> / <code><</code>	больше / меньше
<code>>=</code> / <code><=</code>	больше или равно / меньше или равно

```
python
```

```
hp = 30
print(hp == 30)    # True
print(hp != 0)     # True
print(0 < hp <= 100) # True – в Python сравнения можно объединять в цепочку
print("apple" < "banana") # True – строки сравниваются по алфавиту
```

⚠ Не путай `=` (присвоить) и `==` (сравнить).

3.4. Логические операторы

Оператор	Смысл	Истина, когда...
<code>and</code>	И	оба условия истинны
<code>or</code>	ИЛИ	хотя бы одно истинно
<code>not</code>	НЕ	инвертирует значение

Таблица истинности:

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
python
```

```
has_key = True
door_locked = True
can_enter = has_key or not door_locked    # True
```

Короткое замыкание. `and` и `or` перестают вычислять, как только результат известен:

```
python
```

```
x = 0
if x != 0 and 10 / x > 1:    # деление не выполнится, ошибки не будет
    ...
```

Кроме того, `and` и `or` возвращают **одно из исходных значений**, а не обязательно `True` / `False`:

```
python
```

```
name = "" or "Безымянный" # 'Безымянный' — частый способ задать значение по умолчанию
```

3.5. Операторы принадлежности: `in`, `not in`

```
python
```

```
"меч" in ["меч", "щит"]      # True
"hp" in {"hp": 100}          # True — для словаря проверяются ключи
"z" not in "hello"           # True
```

3.6. Операторы тождественности: `is`, `is not`

`is` проверяет, **один ли это объект в памяти**, а `==` проверяет, **равны ли значения**.

```
python
```

```
a = [1, 2]
b = [1, 2]
print(a == b) # True — значения одинаковые
print(a is b) # False — это два разных объекта
```



Правило: `is` используй **только для `None`** (`if x is None`). Для всего остального — `==` .

3.7. Побитовые операторы (продвинутый уровень)

Работают с отдельными битами целых чисел. В геймдеве их применяют для флагов, масок и слоёв коллизий.

Оператор	Смысл	Пример (<code>a=6</code> → <code>110</code> , <code>b=3</code> → <code>011</code>)
<code>&</code>	побитовое И	<code>6 & 3</code> → <code>2</code> (<code>010</code>)
<code> </code>	побитовое ИЛИ	<code>6 3</code> → <code>7</code> (<code>111</code>)
<code>^</code>	исключающее ИЛИ	<code>6 ^ 3</code> → <code>5</code> (<code>101</code>)
<code>~</code>	инверсия	<code>~6</code> → <code>-7</code>
<code><<</code>	сдвиг влево	<code>1 << 3</code> → <code>8</code>
<code>>></code>	сдвиг вправо	<code>8 >> 2</code> → <code>2</code>

python

```
FLAG_FIRE   = 1 << 0   # 0b001
FLAG_POISON = 1 << 1   # 0b010
FLAG_STUN   = 1 << 2   # 0b100

status = FLAG_FIRE | FLAG_STUN   # включить два флага
print(bool(status & FLAG_STUN))  # True — флаг стоит
status &= ~FLAG_FIRE             # снять флаг
```

3.8. Приоритет операторов (от высшего к низшему)

1. `()` — скобки
2. `**`
3. `+x`, `-x`, `~x` (унарные)
4. `*`, `/`, `//`, `%`
5. `+`, `-`
6. `<<`, `>>`
7. `&`
8. `^`
9. `|`
10. `==`, `!=`, `<`, `>`, `<=`, `>=`, `is`, `in`
11. `not`
12. `and`
13. `or`

python

```
print(2 + 3 * 4)      # 14, а не 20
print((2 + 3) * 4)    # 20
print(-2 ** 2)        # -4 — степень вычисляется раньше унарного минуса
print(True or False and False) # True — and выполняется раньше or
```



Совет: если сомневаешься в порядке, **ставь скобки**. Так код и читается проще.

3.9. Тернарный (условный) оператор

python

```
hp = 0
status = "мёртв" if hp <= 0 else "жив"
```

4. Частые ошибки новичков

Ошибка	Что происходит	Как правильно
<code>if x = 5:</code>	синтаксическая ошибка	<code>if x == 5:</code>
<code>"5" + 5</code>	<code>TypeError</code>	<code>int("5") + 5</code>
<code>0.1 + 0.2 == 0.3</code>	<code>False</code>	<code>math.isclose(...)</code>
<code>b = a</code> для списка	общий объект, а не копия	<code>b = a.copy()</code>
<code>x is 5</code>	ненадёжная проверка	<code>x == 5</code>
<code>int(3.9)</code> ждёт 4	получается 3	<code>round(3.9)</code>
<code>x++</code>	ошибка	<code>x += 1</code>
имя <code>list = [...]</code>	затирается встроенная функция	<code>items = [...]</code>

5. Мини-пример: всё вместе

```
python

# Простая боевая сцена
MAX_HP = 100

player = {"name": "Arsi", "hp": MAX_HP, "attack": 15, "statuses": set()}
enemy_hp = 40
turn = 0

while enemy_hp > 0 and player["hp"] > 0:
    turn += 1
    crit = turn % 3 == 0 # каждый 3-й ход — крит
    damage = player["attack"] * (2 if crit else 1)
    enemy_hp = max(enemy_hp - damage, 0)
    print(f"Ход {turn}: урон {damage}{' (КРИТ!)' if crit else ''}, у врага {enemy_hp} HP")

    if enemy_hp > 0:
        player["hp"] -= 8
        if turn == 2:
            player["statuses"].add("poison")

alive = player["hp"] > 0
print(f"Итог: {'победа' if alive else 'поражение'}, HP = {player['hp']}/{MAX_HP}")
print("Отравлен:", "poison" in player["statuses"])
```

В этом примере есть переменные, константа, `int`, `str`, `bool`, `dict`, `set`, арифметика, `%`, сравнения, `and`, `in`, тернарный оператор и f-строки.

6. Упражнения

1. Создай переменные `a = 17` и `b = 5`. Выведи результат всех семи арифметических операторов.
2. Что выведет `print(10 / 5)`? Почему это не `2`?
3. Переведи 3725 секунд в формат «часы:минуты:секунды» с помощью `//` и `%`.
4. Каким будет результат `bool("False")`?
5. Поменяй местами значения `x` и `y` без третьей переменной.
6. Напиши условие: «число `n` лежит от 1 до 100 включительно и при этом чётное».
7. Что выведет этот код?

```
python
```

```
a = [1, 2]
b = a
b += [3]
print(a)
```

8. Чему равно `2 + 3 * 2 ** 2`?

Ответы

1. 22, 12, 85, 3.4, 3, 2, 1419857
2. 2.0 — оператор `/` всегда возвращает `float`.
3. Решение:

```
python

t = 3725
h, m, s = t // 3600, t % 3600 // 60, t % 60
print(f"{h}:{m:02}:{s:02}") # 1:02:05
```

4. `True` — строка не пустая, а её содержимое значения не имеет.
5. `x, y = y, x`
6. `1 <= n <= 100 and n % 2 == 0`
7. `[1, 2, 3]` — `a` и `b` указывают на один и тот же список, а `+=` меняет список на месте.
8. `14`: сначала `2 ** 2 = 4`, потом `3 * 4 = 12`, потом `2 + 12`.

Итог

- **Переменная** — это имя для значения. `=` означает присваивание.
- **Типы** определяют, что можно делать со значением: числа (`int`, `float`), текст (`str`), логика (`bool`), пустота (`None`), коллекции (`list`, `tuple`, `dict`, `set`).
- Помни про **изменяемость**: со списками, словарями и множествами `b = a` не создаёт копию.
- **Операторы** бывают арифметические, присваивания, сравнения, логические, принадлежности (`in`), тождественности (`is`) и побитовые.
- Порядок вычислений определяет **приоритет**. Если сомневаешься, ставь скобки.



Следующая тема — **условия** (`if/elif/else`) и **циклы** (`for`, `while`): они опираются на всё, что здесь разобрано.